

GITHUB INTENSIVO

Prof: Daniel Castillo → dcastillo@tdtbcn.onmicrosoft.com
Cibernarium

Tipos de control de versiones:

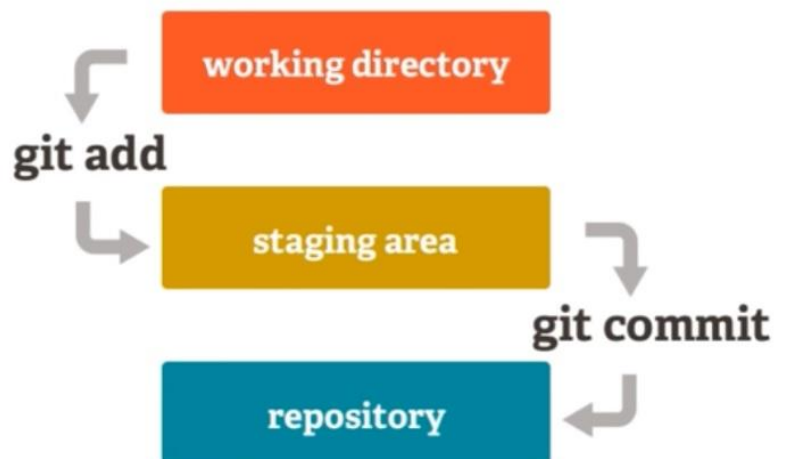
- distribuido
 - peer to peer (cloud) red entre iguales, no hay servidores, ni clientes. No necesitamos un servidor. Está en local. Varios dispositivos tienen copias locales de los repositorios.
- centralizado
 - cuando se utiliza un servidor para controlar y almacenar todos los archivos. Un servidor único

Estados:

1. repositorio (almacén de código/ficheros)
 - a. COMMIT → se hace para confirmar los cambios y pasar al repositorio.
2. staging area (ficheros/código preparados para subir al almacén(repositorio))
3. working directory (área de trabajo) en local

Basic commands

- git init
- git add <file>
- git status
- git commit
- git push
- git pull
- git clone



Tracked (unmodified) → ficheros bajo seguimiento

Untracked → ficheros que no están en seguimiento

Staged → ficheros preparados para ser confirmados

****Para pasar un fichero de una área a otra : Añadidos con add, y confirmamos (cambios) con un commit.**

Bajar GIT:

<https://git-scm.com/download>

Para ver dónde se ubica por defecto los archivos en local:

```
$ pwd
```

Ver usuario: (usuario que permite conectarse con el remoto)

```
git config --global user.name
```

Ver email

```
git config --global user.email
```

Configurar el usuario:

```
git config --global user.name "nombre_usuario" → Configurar Usuario
```

Para configurar el email del usuario que hace el commit

```
git config --global user.email "demo@gmail.com"
```

Ver toda la configuración

```
git config --list
```

Listar el usuario global configurado

```
git config --global --list
```

Editar usuario global

```
$ git config --global --edit
```

Configuraciones (niveles):

1. global
2. local
3. system

Ayuda(help)

Ver una lista de comandos disponibles

```
git help
```

Abre en el navegador un tutorial de git

```
git help tutorial
```

Abre tutorial de git push

```
git help push
```

Abre tutorial de git commit

```
git help commit
```

Crear un directorio (se crea en desktop ya que es donde apunta el pwd)

```
$ mkdir <nombre_carpeta_proyecto> ej:$ mkdir PRUEBAGIT
```

Abrir carpeta(bajar un nivel) → también se puede poner la ruta de la carpeta

```
cd PRUEBAGIT
```

Subir un nivel

```
cd..
```

Seleccionar esta carpeta (PRUEBAGIT) para hacer el control de versiones:

```
git init
```

Crea directamente desde el cd .. (nivel inicial<Desktop>), crea la carpeta y ya la deja preparada para almacenar el control de versiones.

```
git init MIPRUEBA
```

Ver estado del repositorio:

```
git status
```

Hacer un add(para hacer un track)

Por ejemplo, creamos un fichero vacío dentro de la carpeta PRUEBAGIT → el fichero está untracked, si hago un git status detecta el fichero pero me dice que no está añadido, hay que hacerle un git add para decirle que pase el fichero a staged `git add "Fichero1.txt"`; ahora al hacer git status aparece en el staged pero que hay cambios que deben ser comiteados.

****Si modifico el nombre o el contenido de un archivo tengo que hacer un add.**

Add de todos los ficheros con extensión txt:

```
git add "*.txt"
```

Add de todo lo que hay en la carpeta:

```
git add * ó git add .
```

Añade solo los ficheros tracked (los que ya están en seguimiento, los nuevos no):

```
git add -u
```

Deshacer el git add si nos hemos equivocado:

```
git reset → nos devuelve a estado unstaged.
```

Hacer un commit:

```
git commit -m "Mi primer commit"
```

Me dice las diferencias de lo que había antes respecto a lo que hay ahora en el fichero:

```
git diff
```

```
$ git diff
diff --git a/index.txt b/index.txt
index 0d74cb6..43fd325 100644
--- a/index.txt
+++ b/index.txt
@@ -1,3 +1,4 @@ se ha eliminado una línea, la línea 3; se ha añadido una línea, la línea 4
  Capítulo 1: Introducción
  Capítulo 2: Comandos básicos
- Capítulo 3: Gestión de ramas
\ No newline at end of file
+ Capítulo 3: Repositorios Remotos
+ Capítulo 4: Gestión de ramas
\ No newline at end of file
```

Ver todos los commits hechos:

```
git log
```

```
Aliuvys@ojaldre MINGW64 ~/Desktop/PORFIN (segunda)
$ git log
commit 49203e09e93401b72161bd7863a944ba51dde725 (HEAD -> segunda)
Author: yalipct <aliuvys5@gmail.com>
Date:   Fri Dec 3 16:22:56 2021 +0100
    commit a borrar

commit cfa37970ceb20c5f0d5258f8b50d76c3af2b1102 (origin/master, master)
Author: yalipct <aliuvys5@gmail.com>
Date:   Fri Dec 3 15:45:14 2021 +0100
    Mi primer commit inverso
```



identificador del commit

Cambiar mensaje del último commit:

```
git commit --amend -m "Añadir línea 4"
```

Ver cambios entre commit y commit:

```
git show
```

```
$ git show
commit 4a3219aa3a532577cf6f185bd92764ba00a2b398 (HEAD -> master)
Author: yalipct <aliuvys5@gmail.com>
Date:   Wed Dec 1 17:02:25 2021 +0100

    Añadir línea 4

diff --git a/index.txt b/index.txt
index 0d74cb6..43fd325 100644
--- a/index.txt
+++ b/index.txt
@@ -1,3 +1,4 @@
  Capítulo 1: Introducción
  Capítulo 2: Comandos básicos
- Capítulo 3: Gestión de ramas
\ No newline at end of file
+ Capítulo 3: Repositorios Remotos
+ Capítulo 4: Gestión de ramas
\ No newline at end of file
```

Creamos un nuevo fichero llamado index2.txt y hacemos un cambio en index.txt si hago un git status me dice que el `index.txt` está modificado y el `index2.txt` está untracked

```
q enter → volver a la consola
```

Eliminar repositorio local (elimina el `.git` y ya no tiene referencia de dónde almacenar el repositorio en local)

```
rm -rf .git
```

****Puedo hacer un git init pero el repositorio está en blanco tengo que volver a hacer git add . y git commit.**

Crear rama llamada tercera:

```
git branch tercera
```

Crear una rama y nos cambia a ella:

```
git checkout -b segunda
```

Cambiar de rama (ejemplo a segunda):

```
git checkout segunda
```

Listar ramas: → si no has hecho un commit no funciona

```
git branch
```

```
Aliuvys@ojaldre MINGW64 ~/Desktop/BOOK (segunda)
$ git branch
master
* segunda ➡ en color verde la rama en la que estoy
```

Trabajar con el repositorio remoto (github)

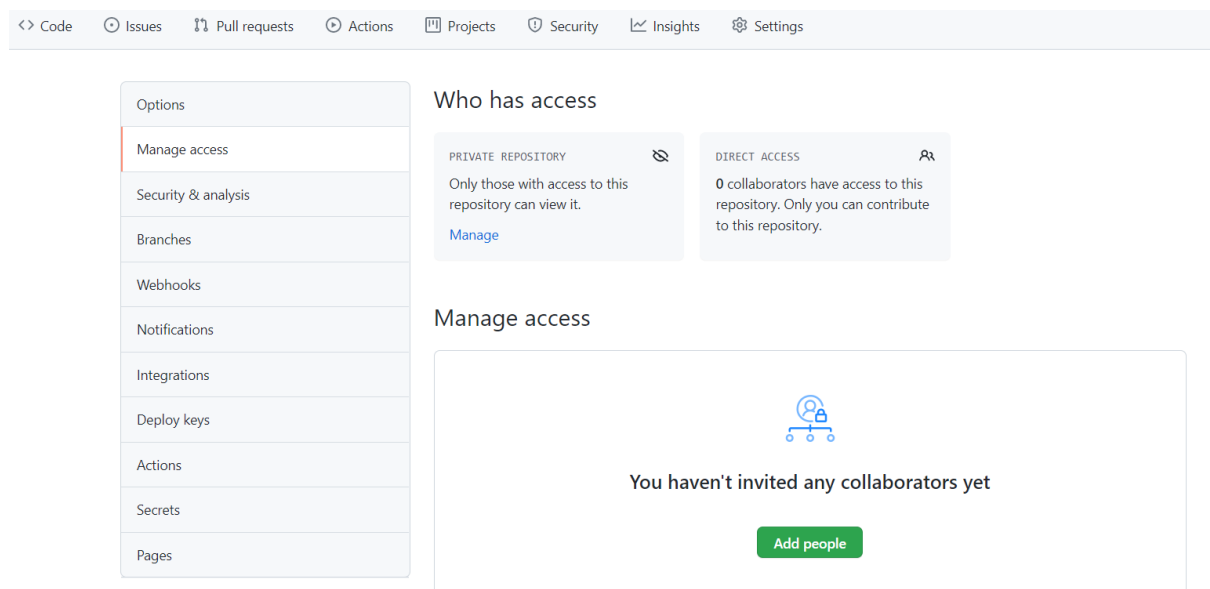
Crearse una cuenta de github, crear un nuevo repositorio ej: CURSOGIT2021

Para clonar en local tengo que estar en la raíz, pwd = mi carpeta raíz

Readme → contendrá las funcionalidades del código (que hace)

Settings/Options/Danger Zone(cambiar visibilidad, eliminar el repositorio)

ManageAccess → enviar solicitud para colaborar en mi proyecto (deben aceptar la colaboración)

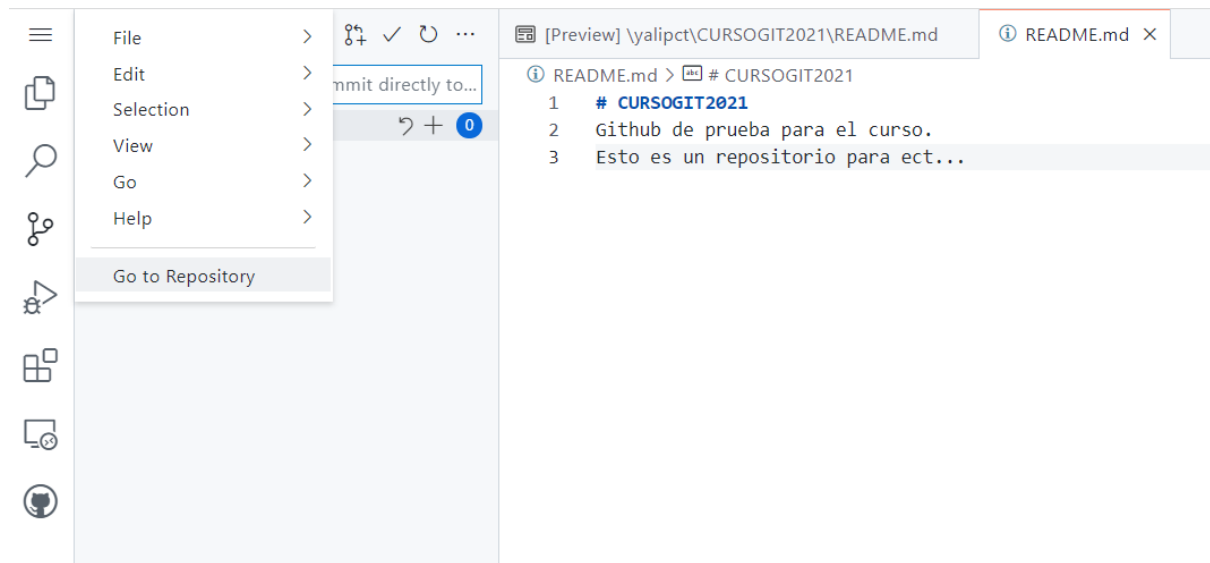


The screenshot shows the GitHub repository settings page. The left sidebar contains a list of options: Options, Manage access (highlighted), Security & analysis, Branches, Webhooks, Notifications, Integrations, Deploy keys, Actions, Secrets, and Pages. The main content area is titled 'Who has access' and shows two sections: 'PRIVATE REPOSITORY' and 'DIRECT ACCESS'. The 'PRIVATE REPOSITORY' section states 'Only those with access to this repository can view it.' and has a 'Manage' link. The 'DIRECT ACCESS' section states '0 collaborators have access to this repository. Only you can contribute to this repository.' Below these sections is a 'Manage access' section with a message 'You haven't invited any collaborators yet' and an 'Add people' button.

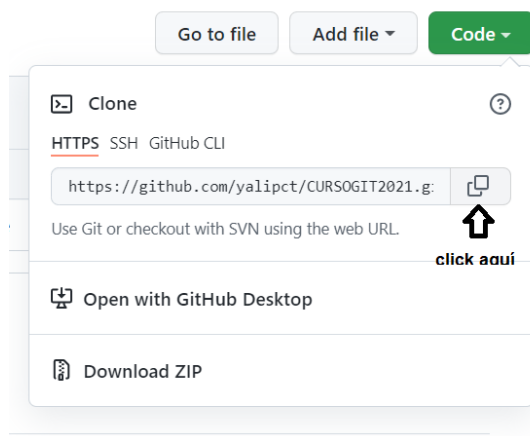
Commit new file → en github hace los dos pasos pulsando el botón(add y commit)

Dentro del repositorio si pulsas punto (.) abre un editor de visual code vía web.

Volver del Visual Web Code al repositorio



Clonar repositorio al local(a través de la consola)



en consola pongo lo siguiente: (seguridad simple → sin SSH)

```
git clone https://github.com/yalipct/CURSOGIT2021.git
```

En local (recomendaciones)

1. crear rama (con nuestro nombre)
2. hacer cambios en el código de los ficheros
3. preparar cambios + confirmar
4. git push <url_remota>
5. git push origin main

Para subir al remoto hay que hacer los dos pasos sgtes:

1. Subir los cambios al remoto: (abre la conexión con el remoto)

```
git push https://github.com/yalipct/CURSOGIT2021.git
```

2. Everything up-to-day (empujar la rama a origin main) → confirma en el local los cambios subidos al remoto

```
git push origin main
```

Si hago cambios en una rama por ejemplo: **segunda**, pasos para subirlo a remoto:

1. git add .
2. git commit -m "mensaje"
3. git push https://github.com/yalipct/CURSOGIT2021.git
4. Puedo empujar a **segunda** o **main**
 - a. empujo a main → git push origin main (no es lo correcto el main no se toca)
5. En el remoto hago el Compare & pull request
6. Hacer merge (hay 3 opciones) de momento Create a merge commit (que pone por defecto)
7. Hago el Merge pull request que es como otro comentario
8. Borrar la rama **segunda**.

Corregir lo de arriba para hacerlo de manera correcta:

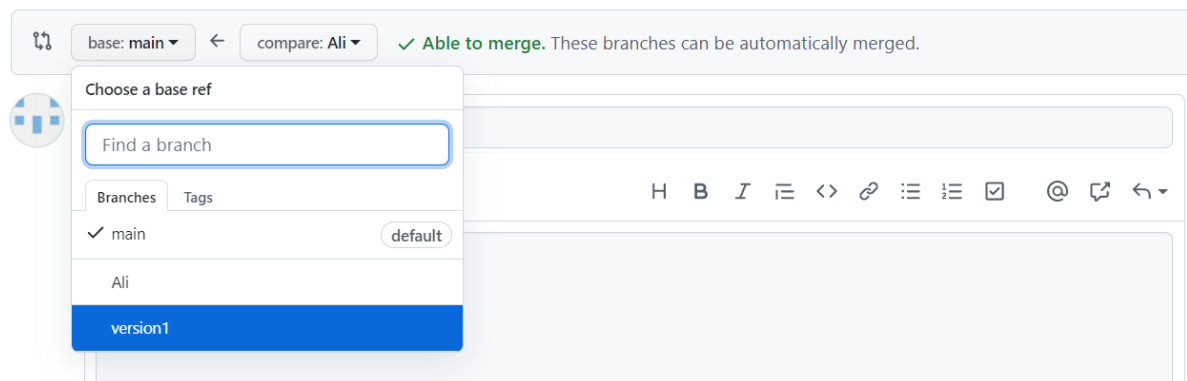
1. Modificar el fichero
2. Borrar del escritorio la carpeta del repositorio PRACTICA1 (en local)
3. Cerrar la consola
4. No hacer Pull Request (en remoto)
5. Copiar url del repositorio para clonar
6. En Git Bash clonar solo la rama version1

```
$ git clone --branch version1 https://github.com/yalipct/PRACTICA1.git
```

7. Crear una rama con mi nombre ej: Ali
8. git add *
9. git commit -m "mensaje"
10. git push <https://github.com/yalipct/PRACTICA1.git>
11. git push origin version1
12. hacer pull request pero ojo tiene que seleccionar base:version1 compare:Ali

Open a pull request

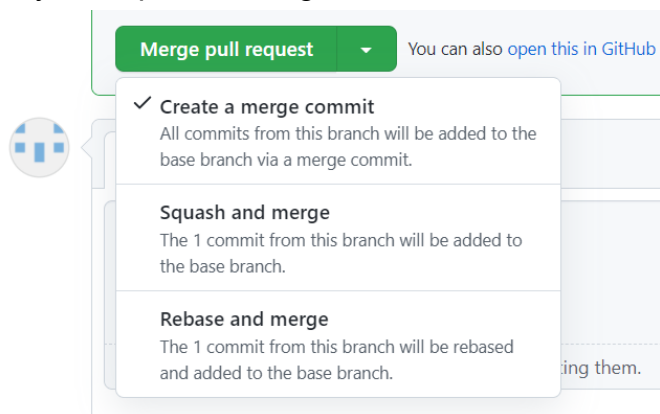
Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).



13. mergear

14. borrar la rama Ali que uso como pasarela de los cambios.

Hay tres tipos de merge:



create a merge commit: es el normal (añade todos los commits individualmente, si tienes 10 sube los 10)
squash and merge: compacta todo en un solo commit
rebase and merge: hace también un solo commit pero en una nueva rama.

15. voy a hacer un cambio en remoto, eliminar la rama en local Ali para bajarme los cambios desde el remoto:

- vuelvo a la rama version1 → `git checkout version1`
- borro la rama Ali → `git branch -D Ali`

16. Hago un git fetch para informar los cambios al local.

17. Hacer una git merge para traerme los cambios.

Bajar cambios de remoto:

```
git fetch → informa de los cambios al local  
git merge → incluye los cambios en local
```

Eliminar una rama:

```
git branch -D nombre_dela_rama
```

Clonar solo una rama:

```
$ git clone --branch version1  
https://github.com/yalipct/PRACTICA1.git
```

Antes de subir las nuevas modificaciones hay que hacer un pull para traer los cambios del remoto al local:

```
git pull https://github.com/yalipct/CURSOGIT2021.git
```

Para actualizar del remoto al local

```
git fetch + git merge  
git pull
```

Trabajar en un proyecto público en paralelo (si haces un fork de cortesía le puedes dar Star ☆)

Fork → clonar un repositorio público para integrarlo en mi github ("hacerlo mío").

Hace una copia exacta en crudo (en inglés «bare») del repositorio original que podemos utilizar como un repositorio git cualquiera. (lo pone en nuestro repositorio)

Ejercicio:

PROYECTO DICIEMBRE 1(team creado por el profesor)

- clonar el proyecto en local
- añadir un fichero con una o varias líneas de códigos
- modificar el fichero index.txt
- subir los cambios al remoto
 - ◆ IMPORTANTE: el nombre de la rama + commit con lo que has hecho
- tenemos que actualizar nuestro repositorio local
 - ◆ cambiar a la rama main
 - ◆ eliminar la rama con nuestro nombre (Ali)
 - ◆ git fetch + git merge

Después del fetch + merge anterior:

Si hago un git branch solo veré la rama main

Para ver todas las ramas:

```
git branch -a → permite tener acceso a otras ramas
```

Para cambiar de rama:

```
git checkout isabel → escojo la rama que quiero
```

Proyecto en local subir al github:

Subir repositorio local al remoto:

Instrucciones:

crear un repositorio remoto en github (PORFINVIERNES)

crear una carpeta en el escritorio (PORFIN)

copiar la url del repositorio remoto

crear dos ficheros .txt (vacíos) dentro de la carpeta PORFIN

Pasos para subir los ficheros al remoto:

1. Git bash here en la carpeta creada en local (PORFIN)
2. git init
3. git add .
4. git commit -m "mensaje"
5. git remote add origin <https://github.com/yalipct/PORFINVIERNES.git>
6. git push -u origin master

****Si cuando creo el repositorio añado el Readme me crea la rama **main**, si no crea la rama es **master**.**

Si quiero cambiar o **renombrar** la rama a main (o la que sea, por ejemplo en caso de que equivoque) antes de hacer el git push:

```
git branch -M main
```

Deshacer cambios que hemos hecho hasta este momento en un fichero:

```
git checkout -- Fichero1.txt
```

Deshacer todos los cambios hechos después del último commit:

```
git checkout -- *
```

Eliminar el último commit hecho:

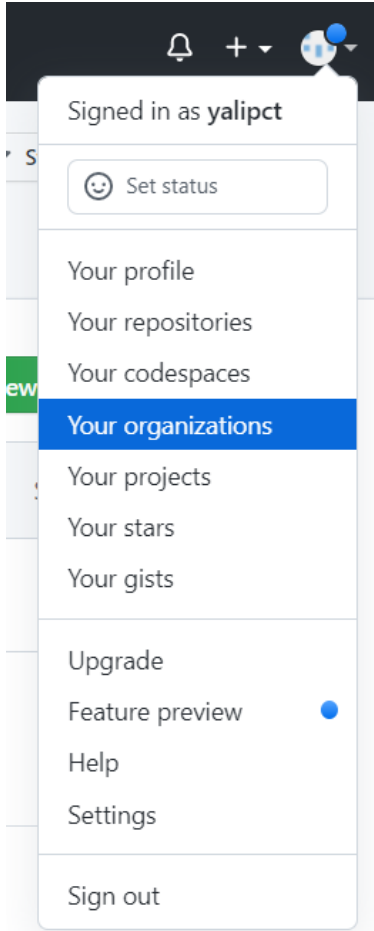
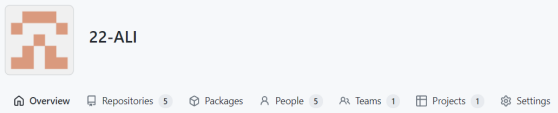
```
git reset HEAD^ --hard
```

GitHub Desktop:

Abrir consola (prompt):

menú Repository/Open in Command Prompt

Ejercicio crear una organización(en remoto):

	Una vez dentro: New organization Seleccionar Create free organization y completar los datos. Una vez creada dentro de la organización tendremos el sgte menú:  Repositories: podemos crear repositorios como los que nos piden más abajo. People: podemos añadir miembros a la organización. Teams: podemos crear equipos de trabajo o añadir miembros a un equipo ya creado. Projects: Podemos crear un proyecto o ver las fases de un proyecto ya creado. (Kanban básico → ver imagen)
--	---

Un proyecto para TICKETING-DU-SOLEIL

En la vista Repositories creo los sgtes repositorios:

REPOSITORIO 1: TRABAJAR CON LA BBDD

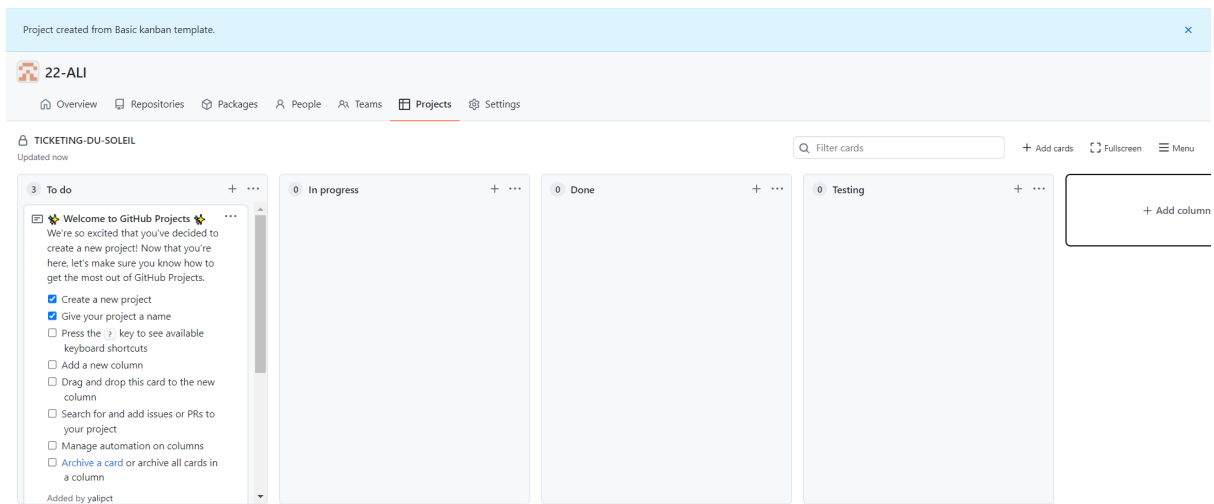
REPOSITORIO 2: DESARROLLO FUNCIONES GENÉRICAS

REPOSITORIO 3: INTERFAZ GRÁFICA

REPOSITORIO 4: DISEÑO DE LA APP

REPOSITORIO 5: INTEGRACIÓN CODIGO FINAL

Trabajar en un proyecto con github, Kanban básico (fases de un proyecto):



Organización

Podemos crear equipos, crear repositorios, crear issues, asignar personas, asignar labels, asignar el proyecto, asignar milestone...

Milestone: grupo de tareas que tiene tiempo de finalización y progresión del porcentaje realizado (varias tareas hacen un 100%)